New Selection Algorithm Based on Generating Sequences

Y. N. Artamonov[1], Y. V. Alyaeva[2]

[1]Federal State Budgetary Scientific Institution «Gosmetodcenter»
[2]National Research University «Moscow Power Engineering Institute»
Russia

Международная конференция
«Динамические системы и компьютерные науки:
теория и приложения» (DYSC 2020)
Иркутск, Россия, 19-21 сентября 2020 г.

Формулировка задачи, краткий обзор существующих подходов

Общая формулировка

Дано множество X , отношение порядка $\prec$ на X и целое число k. Требуется найти k-й наименьший элемент, т.е. элемент $x \in X$, для которого существует по крайней мере $(k-1)$ элементов $\forall j = \overline{1, k-1} : x_j \prec x$ и не менее k элементов $\forall j = \overline{1, k} : x_j \prec x \vee x_j = x$.
Введем обозначение $x = Q(X, k)$, тогда $Q(X, 1) = \min(X), Q(X, |X|) = max(X), Q\left(X, \lceil \frac{|X|}{2} \rceil\right) = median(X)$

Алгоритм FIND - quickselect (Хоар)

```
1: function QUICKSELECT(X, k)
2:     if k = 1 ∧ |X| = 1 then
3:         return X[1]
4:     else
5:         L ← ∅, R ← ∅, LL = 0, P ← X[Rand(|X|)]
6:         while i ≤ |X| do
7:             if X[i] < P then
8:                 Push(X[i], L), LL ← LL + 1
9:             else
10:                Push(X[i], R)
11:            end if
12:            i ← i + 1
13:        end while
14:        if k ≤ LL then
15:            return quickselect(L, k)
16:        else
17:            return quickselect(R, k − LL)
18:        end if
19:    end if
20: end function
```

Вычислительная сложность
Пусть $n = |X|$, обозначим $C(n, k)$ - количество сравнений при нахождении $Q(X, k)$. Доказано (**Knuth D E 1973 The Art of Computer Programming. Sorting and Searching**), что:

$$C(n,k) = 2 \cdot ((n+1) \cdot H_n - (n-k+3)H_{n-k+1} - (k+2)H_k + n + 3), H_k = \sum_{i=1}^{k} \frac{1}{i}$$

В частых случаях:

$$C(n, 1) = C(n, n) = 2n + o(n)$$

$$C\left(n, \lceil \frac{n}{2} \rceil\right) = 3.39n + o(n)$$

Алгоритм SELECT

```
1: function SELECT(X, k)
2:     if n = 1 then
3:         return X[1]
4:     else
5:         s ← Rand(n), g > 0                          ▷ Инициализация
6:         S = sample(X), |S| = s                      ▷ Выбор образца
7:         i_u = max{1, ⌈ks/n − g⌉}, i_v = min{s, ⌈ks/n + g⌉},
8:         u = Select(S, i_u), v = Select(S, i_v)      ▷ Выбор осей
9:         L ← ∅, U ← ∅, M ← ∅, V ← ∅, R ← ∅
10:        for i = 1, n do
11:            L = {X[i] ∈ X : X[i] < u}
12:            U = {X[i] ∈ X : X[i] = u}
13:            M = {X[i] ∈ X : u < X[i] < v}
14:            V = {X[i] ∈ X : X[i] = v}
15:            R = {X[i] ∈ X : v < X[i]}
16:        end for                                     ▷ Разделение
17:        if |L| < k ≤ |L| + |U| then
18:            return u
19:        else if |L| + |U| + |M| < k ≤ n − |R| then
20:            return v
21:        end if                                      ▷ Тест остановки
22:        if k ≤ |L| then
23:            return Select(L, k)
24:        else if k > n − |R| then
25:            return Select(R, k − n + |R|)
26:        else
27:            return Select(M, k − |L| − |U|)
28:        end if                                      ▷ Редукция
29:    end if
30: end function
```

Статьи:

- Krzysztof C. Kiwiel, On Floyd and Rivest's SELECT algorithm, Theoretical Computer Science. Volume 347, Issues 1–2 (2005)
- R.W. Floyd, R.L. Rivest, Expected time bounds for selection, Comm. ACM 18 (1975)
- R.W. Floyd, R.L. Rivest, The algorithm SELECT—for finding the ith smallest of n elements (Algorithm 489), Comm. ACM 18 (1975)

$$C(n,k) = n + \min(n-k, k) + o(n)$$

Рассмотренные алгоритмы предполагают использование случайного выбора. Для специально подобранных входных данных рассмотренные алгоритмы не гарантируют линейного времени нахождения k-й статистики.

Краткий обзор существующих подходов

Алгоритм PICK (медиана медиан)

Статья:
M.R. Blum, R.W. Floyd, V.R. Pratt, R.L. Rivest, R.E. Tarjan, Time bounds for selection, J. Comput. System Sci. 7 (1972)
Предложена модификация алгоритма quickselect - алгоритм PICK, гарантирующий $O(n)$ для худшего случая. Таким образом, доказана теоретическая возможность линейного времени нахождения k-го наименьшего элемента в любом случае. Однако, верхнее ограничение, доказанное в статье составляет не более $5.4305n$ сравнений. Также в статье обосновано нижнее ограничение на нахождение медианы не менее $\frac{3}{2}n - 2$ сравнения.

Основной замысел предлагаемого алгоритма

Определения, доказанные результаты

Статья: Artamonov, Y.N. (2017). Using the Complete Sequences for Sorting Natural Numbers. The Bulletin of Irkutsk State University. Series Mathematics. 22. 3-17. doi.org/10.26516/1997-7670.2017.22.3

Определение

Неубывающая бесконечная последовательность натуральных чисел $V = [v_1, v_2, v_3, \ldots]$ *называется полной, если каждое натуральное число можно представить в виде суммы некоторых чисел из* V*:* $\forall n \in \mathbb{N} : n = \sum_{i=1}^{\infty} e_i \cdot v_i, \forall i : e_i \in \{0, 1\}$.

Теорема (теорема Д. Брауна)

Для того, чтобы неубывающая последовательность $V = [v_1, v_2, v_3, \ldots]$ *была полной, необходимо и достаточно, чтобы:*

1. $v_1 = 1$;
2. $\forall k > 1 : v_k - 1 \leq v_1 + v_2 + \ldots + v_{k-1}$.

Определение

Для заданного натурального числа n *назовем конечную последовательность натуральных чисел* $B = [b_1, b_2, \ldots, b_m], b_i \geq b_{i+1}$ *порождающей, если любое натуральное число* $k < n$ *можно представить в виде суммы элементов этой последовательности:* $\forall k < n : k = \sum_{i=1}^{m} e_i \cdot b_i, e_i \in \{0, 1\}$*, причем соблюдается условие:* $n = \sum_{i=1}^{m} b_i$.

Исходя из данного определения, первые m элементов любой полной последовательности, записанные в обратном порядке, составляют порождающую последовательность для числа $n = \sum_{i=1}^{m} v_i$.

Основной замысел предлагаемого алгоритма

Определения, доказанные результаты

```
1: function CAlgorithm(n, c)
2:     a ← n, i ← 1
3:     while a ≠ 0 do
4:         B[i] ← ⌈a/c⌉
5:         a ← a − B[i]
6:         i ← i + 1
7:     end while
8:     return B
9: end function
```

Лемма

*Для любого натурального числа n и любого вещественного положительного числа c, такого, что $\lceil \frac{n}{c} \rceil \leq \frac{n+1}{2}$ последовательность чисел b_i, получаемая по алгоритму **CAlgorithm** является порождающей последовательностью длины $m \leq \lfloor \log_{\frac{c}{c-1}} n \rfloor + 1$.*

Лемма

*Порождающая последовательность чисел b_i, получаемая по алгоритму **CAlgorithm** для заданного числа n при $c = 2$ имеет минимальную длину.*

Лемма

*Пусть для заданных n и c, таких, что $\lceil \frac{n}{c} \rceil \leq \frac{n+1}{2}$, по алгоритму **CAlgorithm** получена порождающая последовательность $B = [b_1, b_2, \ldots, b_m]$, тогда всегда существует не менее одного, но не более двух натуральных чисел $n' > n$, для которых порождающая последовательность по алгоритму **CAlgorithm** будет $B' = [b_0, b'_1, b'_2 \ldots, b'_m]$ такая, что $\forall i = 1..m : b_i = b'_i$.*

Определения, доказанные результаты

Теорема

Выбрав произвольное натуральное число n, а также число c из условия $\lceil \frac{n}{c} \rceil \leq \frac{n+1}{2}$, можно сформировать последовательность $A = [a_1, a_2, \ldots, a_k, \ldots, a_{m+1}], a_{m+1} = n, a_{k-1} = a_k - \lceil \frac{a_k}{c} \rceil, a_2 = 1, a_1 = 0$ с последующим добавлением в нее элементов до бесконечности:

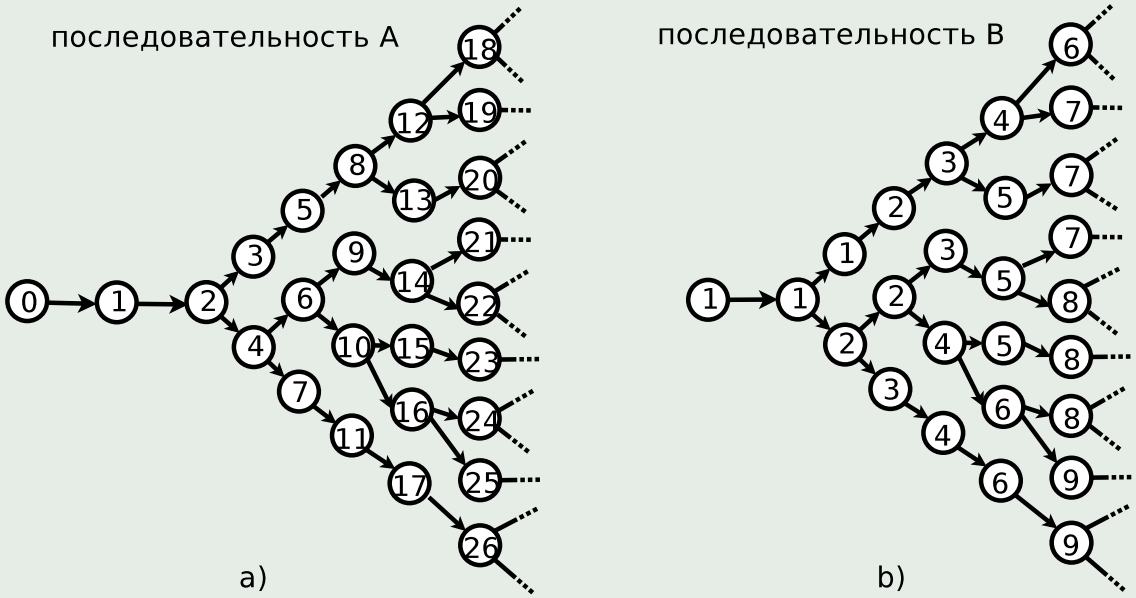
$$n' = \lceil \frac{nc}{c-1} \rceil \vee n' = \lceil \frac{nc+1}{c-1} \rceil, A' = [a_1, a_2, \ldots, a_k, \ldots, a_{m+1}, n']$$

$$\ldots$$

$$n^{(k)} = \lceil \frac{n^{(k-1)}c}{c-1} \rceil \vee n^{(k)} = \lceil \frac{n^{(k-1)}c+1}{c-1} \rceil,$$

$$A^{(k)} = [a_1, a_2, \ldots, a_k, \ldots, a_{m+1}, n', \ldots, n^{(k)}]$$

и т.д. Тогда последовательность B' первых разностей от $A^{(k)}$ будет полной последовательностью: $B' = [b'_1, b'_2, \ldots, b'_{m+k}, \ldots], \forall i : b'_i = a_{i+1} - a_i$.



Алгоритм cs-select

```
1: function ITER-FIND(X, B, A, k)        ▷ Вспомогательная функция
2:     if |B| = ∅ or k ≤ 0 or |X| = 1 then
3:         return X[1]
4:     end if                                  ▷ Базы рекурсии
5:     Less ← ∅, More ← ∅, len = 0, A ← A + B[1], i ← 1
6:     while i ≤ |X| do                        ▷ Разделение
7:         if X[i] ≥ A then
8:             push(X[i], More)
9:         else
10:            push(X[i], Less), len ← len + 1
11:        end if
12:        i ← i + 1
13:    end while                               ▷ Рекурсия
14:    if len ≥ k then
15:        return iter-find(Less, B\B[1], A − B[1], k)
16:    else
17:        return iter-find(More, B\B[1], A, k − len)
18:    end if
19: end function
20: function CS-SELECT(X, B, k)
21:    return iter-find(X, B, 0, k)
22: end function
```

Анализ вычислительной сложности

Для простоты оценки изменений мощности множеств $X, Less, More$ примем $c = 2$ и будем считать, что на вход алгоритма cs-select в качестве X подается некоторая перестановка элементов множества: $\{i : i \in \mathbb{N}, i \leq n\}$

Тогда относительно каждого элемента порождаемой последовательности B множества $X, Less, More$ разделяются примерно пополам, и количество сравнений, выполняемых в пункте 7 алгоритма cs-select, можно оценить как:

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor + 1} \lfloor \frac{n}{2^i} \rfloor \leq C(n,k) \leq \sum_{i=0}^{\lfloor \log_2 n \rfloor + 1} \lceil \frac{n}{2^i} \rceil$$

$$n \to \infty \Rightarrow \sum_{i=0}^{\lfloor \log_2 n \rfloor + 1} \lfloor \frac{n}{2^i} \rfloor \to 2n, \sum_{i=0}^{\lfloor \log_2 n \rfloor + 1} \lceil \frac{n}{2^i} \rceil \to 2n$$

$$n \to \infty \Rightarrow C(n,k) \to 2n$$

Оценка вычислительной сложности алгоритма cs-select для разных k, c на перестановках элементов при $n = 100$

n	k	c=2 S=(50,25,13,6,3,2,1)	c=2.5 S=(40,24,15,9,3,2,1,1)	S=(51,26,13,7,4,2,1)
		$C(n,k)/n$	$C(n,k)/n$	$C(n,k)/n$
100	1	1.92	1.9	1.96
	25	1.98	1.89	1.98
	50	2.0	2.16	1.92
	75	2.01	2.27	1.99
	100	2.03	2.51	1.95

Экспериментальные сравнения cs-select, quickselect

The average number of comparisons for sequences of different lengths, obtained on the basis of the cs-select algorithm.

n	$Q(X,1)$	$Q(X,\lceil 0.25n \rceil)$	$Q(X,\lceil 0.5n \rceil)$	$Q(X,\lceil 0.75n \rceil)$	$Q(X,n)$
10	18.65	22	24.15	23.84	23.61
100	197.52	206.02	210.05	206.79	207.06
1000	2000.58	2024.16	2031.53	2010.55	2005.54
10000	19994.11	20034.36	20092.17	20043.37	20014.85
100000	200004.34	200154.56	200243.28	200135.17	200062.48
1000000	2000050.3	2000328.90	2000714.60	2000658.80	2000136.60

The average number of comparisons for sequences of different lengths, obtained on the basis of the quickselect algorithm.

n	$Q(X,1)$	$Q(X,\lceil 0.25n \rceil)$	$Q(X,\lceil 0.5n \rceil)$	$Q(X,\lceil 0.75n \rceil)$	$Q(X,n)$
10	20.58	31.93	31.33	29.5	21.19
100	206.69	309.91	323.55	325.08	206.70
1000	2075.94	2974.03	3369.36	3188.59	1940.85
10000	20404.60	33116.61	34515.53	31773.43	19332.77
100000	210805.88	308239.38	348473.56	311948.80	197159.25
1000000	2025686.80	3172760.50	3371014.50	3138005.80	1982103.80

Дальнейшие направления исследований

Заметим, что порождающие последовательности можно получать не только в случае выполнения условия $\lceil \frac{n}{c} \rceil \leq \frac{n+1}{2}$.

Например, если взять $c = \frac{1+\sqrt{5}}{2}$, $n = F_k - 1$, где F_k - k-е число Фибоначчи, то порождающей последовательностью можно считать: $B = [F_{k-1}, F_{k-2}, \ldots, F_1]$. Имеем $F_{k-1} = \lceil \frac{F_k - 1}{c} \rceil = \lceil \frac{F_k}{c} - \frac{1}{c} \rceil$, поскольку, как известно $\lim_{k \to \infty} \frac{F_k}{F_{k-1}} = \frac{1+\sqrt{5}}{2}$. Тогда исходная последовательность чисел:

$$T = [1, 2, 3, \ldots, F_{k-2}, F_{k-2}+1, \ldots, F_{k-1}-1, F_{k-1}, F_{k-1}+1, \ldots, F_k - 1],$$

с учетом $F_k - 1 - F_{k-1} = F_{k-2} - 1$, преобразуется к виду:

$$T' = [1, 2, 3, \ldots, F_{k-2}, F_{k-2}+1, \ldots, F_{k-1}-1, 0, 1, \ldots, F_{k-2} - 1]$$

с максимальным числом $F_{k-1} - 1$. Поэтому для следующих шагов данные преобразования повторяются (действительно, как известно, числа Фибоначчи образуют полную последовательность).

Получение порождающих последовательностей при $c < 2$

Теорема

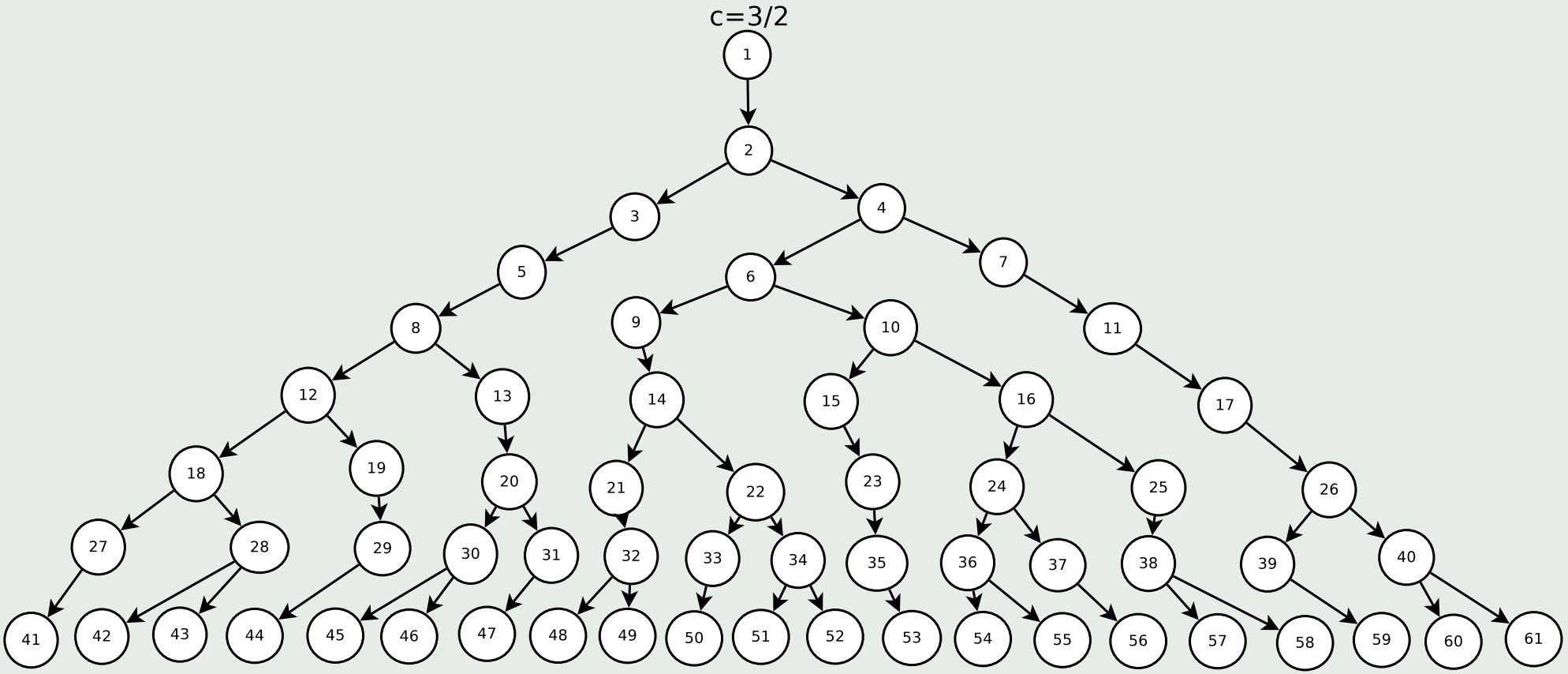
Для любых $b_1 \in \mathbb{N}, 1 < c < 2$ *последовательность* $B = \{b_1, b_2, \ldots, 1\}$, *получаемая по алгоритму:* $b_i = \lceil \frac{b_{i-1}-1}{c} \rceil$ *является порождающей последовательностью.*

Теорема

Пусть $B = \{b_1, b_2, \ldots, b_n\}$ *порождающая последовательность для некоторого* $1 < c < 2$, *которая получается из* b_1 *по правилу:* $b_i = \lceil \frac{b_{i-1}-1}{c} \rceil, 1 < i \leq n$. *Требуется найти такое* b_0, *что порождающая последовательность* $B' = \{b_0, b_1, b_2, \ldots, b_n\}$ *получается из* b_0 *по такому же правилу* $b_1 = \lceil \frac{b_0-1}{c} \rceil$.
Тогда, если $c \cdot (b_1 - 1) \in \mathbb{N}$, *то* $b_0 = \lfloor c \cdot b_1 + 1 \rfloor = \lceil c \cdot b_1 \rceil$.
Если $c \cdot (b_1 - 1) \notin \mathbb{N}$, *то* $b_0 = \lceil c \cdot (b_1 - 1) + 1 \rceil$ *или* $b_0 = \lfloor c \cdot b_1 + 1 \rfloor$.

Дальнейшие направления исследований

Деревья порождающих последовательностей при $1 < c < 2$



A061419

A081226

Дальнейшие направления исследований

Идея сжатия бинарных последовательностей

Пусть $L_2 = \{1, 2, 4, 8, 16, 32, 64, \ldots\}$,
Рассмотрим полную последовательность

$$L = \{1, 2, 4, 7, 12, 21, 36, 61, 104, 117, 301, \ldots\},$$

а также бинарную последовательность $B = 10011000010$ длины 11, которая соответствует числу 400 по жадному алгоритму для L. Бинарное представление числа 400 с помощью L_2 соответствует последовательности $B' = 110010000$ длины 9. Т.е. мы сократили последовательность на 3 бита.